



SWEN 221



Marco Servetto
CO258

www.youtube.com/MarcoServetto

Old Java and new Java goes to battle



New Java

Smaller code base

Less examples online

Main weapons:

- Interfaces
- Records
- Lambdas
- Streams
- Optionals
- unmodifiable collections

Old Java

Bigger code base

More examples online

Main weapons:

- (abstract) classes
- (super) constructors
- explicit fields
- primitive types
- nulls
- annotations
- reflection



New Java is fighting a harsh battle against its culturally established older self.

**However, it is winning.
Every year, more and more
this new style is
gaining momentum.**

**However, the old Java
is bound to stay
(for retro-compatibility)**

So we have to learn all those features too.



In this lecture:

- Many of the older features of Java
- Interactions with the new features

Useful learning:

- Both new and old Java features are super common in other languages



Traditional Java Classes

```
class Point{  
    private double x;  
    private double y;  
  
    public Point(double x, double y){  
        this.x= x; this.y= y;  
    }//Shadowing, so we must use 'this'  
  
    public double x(){ return x; }  
    public void x(double v){ x = v; }  
  
    public double y(){ return y; }  
    public void y(double v){ y = v; }  
}
```

- Point class.
- Note the private fields, the getters, the setters and the constructor.
- Constructor parameters are **hiding/shadowing** the fields. So we have to write 'this.x' to initialize the field x.

Traditional Java Classes

```
class Point{
    private double x;
    private double y;

    public Point(double x, double y){
        this.x= x; this.y= y;
    }
    //Recent naming conventions
    public double x(){ return x; }
    public void x(double v){ x = v; }

    public double y(){ return y; }
    public void y(double v){ y = v; }
}
```

- Recently the naming conventions for Java are changing.
- Now it is preferred to call getters and setters with the name of the field.

Traditional Java Classes

```
class Point{  
    private double x;  
    private double y;  
  
    public Point(double x, double y){  
        this.x= x; this.y= y;  
    }  
    //Legacy naming conventions  
    public double getX(){ return x; }  
    public void setX(double v){ x = v; }  
  
    public double getY(){ return y; }  
    public void setY(double v){ y = v; }  
}
```

- Recently the naming conventions for Java are changing.
- In the past, the more verbose get-set prefixes were used.

Traditional Java sub-classing

```
class Point3D extends Point{  
    private double z;  
    public Point3D(double x, double y, double z){  
        super(x,y);  
        this.z= z;  
    }  
    public double z(){ return z; }  
    public void z(double v){ z = v; }  
}
```

- This is the correct way to extend a Point class into a Point3D class adding a field.
- Note the use of 'super'.
Now I'm going to show you a range of broken solutions that many students attempts.

```

class Point3D extends Point{//WRONG WAY n.1
    private double z;
    public Point3D(double x, double y, double z){
        this.x= x;//Surface error: the field Point.x is not visible
        this.y= y;//real error: missing super-constructor
        this.z= z;
    }
    public double z(){ return z; }
    public void z(double v){ z = v; }
}

```

- Typically, instead of trying to learn the language features (super in this case), the human programmer tries to wing it, and so tries to simply initialize the fields directly.
- The error they get: Point.x is not visible.
- The human remembers ‘public field = bad’ (without any idea of why)
- The human tries to ‘import’ the fields by repeating them

```

class Point3D extends Point{//WRONG WAY n.2
    private double x;
    private double y;
    private double z;
    public Point3D(double x, double y, double z){
        this.x= x;//implicit super constructor Point() is undefined.
        this.y= y;//Must explicitly invoke another constructor
        this.z= z;
    }
    public double z(){ return z; }
    public void z(double v){ z = v; }
}

```

- Still does not compile:
- The error is even more misleading: constructor Point() is undefined
- Then, the human will add the Point() constructor by editing the code of point.
- The human may not think about ‘why’ we are asked to have a Point() constructor, the human wants to get his code to work, not to learn the language

```

class Point{
    public Point(){}//ADDED LINE
    //as before
    public double x(){ return x; }
    ..
    public double y(){ return y; }
}
class Point3D extends Point{//WRONG WAY n.3
    private double x;
    private double y;
    private double z;
    public Point3D(double x, double y, double z){
        this.x= x;//compiles, but does not do what we wanted
        this.y= y;
        this.z= z;
    }
    public double z(){ return z; }
    public void z(double v){ z = v; }
}

```

- Now, it compiles, but it is still broken!
x() and y() would always just return 0!
- Repeating the fields is **hiding** them, not 'importing them'

```

class Point3D extends Point{//WRONG WAY n.4
    private double x;
    private double y;
    private double z;
    public Point3D(double x, double y, double z){
        this.x= x;
        this.y= y;
        this.z= z;
    }
    public double x(){ return x; }
    public void x(double v){ x = v; }

    public double y(){ return y; }
    public void y(double v){ y = v; }

    public double z(){ return z; }
    public void z(double v){ z = v; }
}

```

- **The ultimate abomination**
- It compiles, and passes many tests
- The human programmer has no idea why it is working now.
- A Point3D declared in this way has 5 fields.

```

//Object Layout:
//Point.x//constantly at zero
//Point3D.x
//Point.y//constantly at zero
//Point3D.y
//Point3D.z

```

Again, the correct version

```
class Point3D extends Point{  
    private double z;  
    public Point3D(double x, double y, double z){  
        super(x,y);  
        this.z = z;  
    }  
    public double z(){ return z; }  
    public void z(double v){ z = v; }  
}
```

- To subclass you **must use super**.
- If you do not have a 'special' call at the start of the constructor, you are already implicitly calling super.
- Precise initialization order:
 - every field can be logically initialized once and only once

Super constructors: all about field initialization

```
class Point{
    private double x;
    private double y;

    public Point(double x, double y){
        this.x= x; this.y= y;
    }
    ...
}
class Point3D extends Point{
    private double z;
    public
    Point3D(double x, double y, double z){
        super(x,y);
        this.z= z;
    }
    ...
}
```

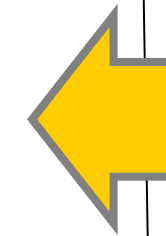
- Point is responsible to initialize x and y. Point3D may not even know x, y.
- Point3D is responsible to instruct Point on how to perform its inner initialization. Point3D is responsible to initialize z.
- ‘super’ has to be the first thing.
- ‘this’ can only be used after the super call. In this way, a Point3D always sees the part of the object that is shared with ‘Point’ as fully initialized.

Inheritance + Constructors

- Constructors are **not** inherited
- Constructors use **super** in first line to forward construction to super class
- If the programmer does not explicitly write the super call, this call is **added by the compiler**.

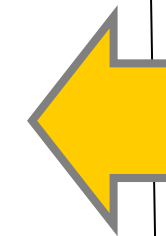
Sneaky code inserted by Java

```
class A { }  
class B extends A {  
    B(){  
        System.out.println("B constr");  
    }  
}
```



How your code looks like

```
class A extends Object { A(){super();} }  
class B extends A {  
    B(){  
        super();  
        System.out.println("B constr");  
    }  
}
```



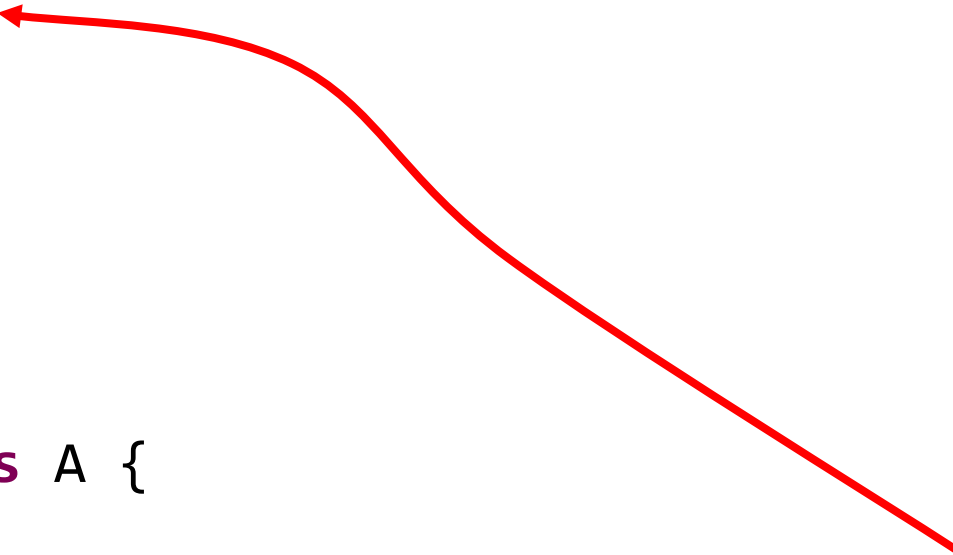
What is added
by Java

Quiz: does it compiles?

```
class A {  
    int f;  
    A(int f){  
        this.f=f;  
    }  
}
```

```
class B extends A {  
    B(){  
        System.out.println("B constr");  
    }  
}
```

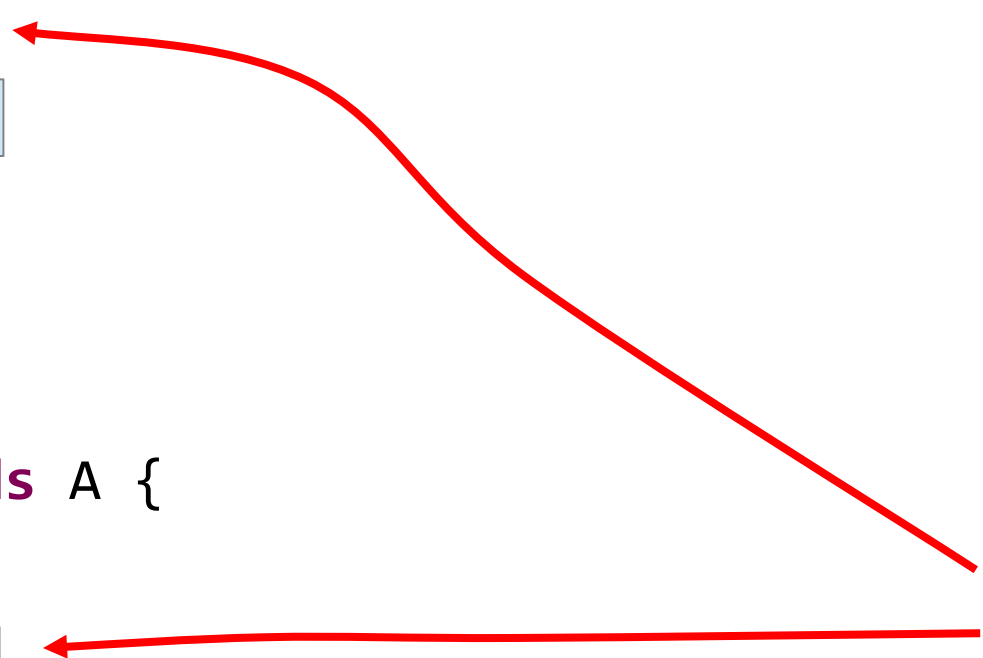
Constructor of A
takes 1 parameter



Quiz: does it compile? NO!

```
class A extends Object {  
    int f;  
    A(int f){  
        super();  
        this.f=f;  
    }  
}  
  
class B extends A {  
    B(){  
        super();  
        System.out.println("B constr");  
    }  
}
```

Constructor of A
takes 1 parameter



Yes,
the language will automatically insert code that does
not even typecheck!

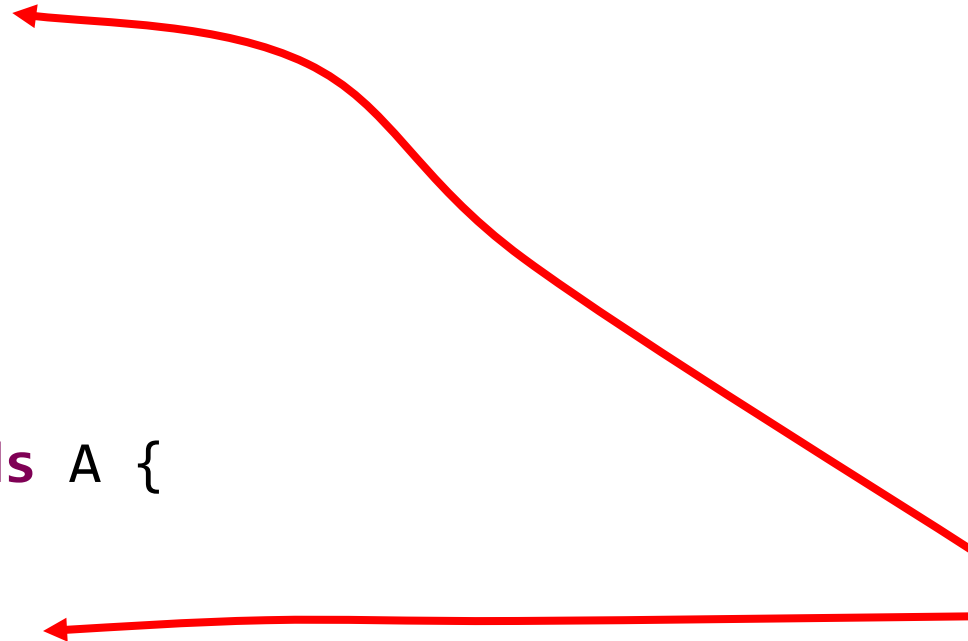
Java hates you.

Possible fix 1

```
class A {  
    int f;  
    A(int f){  
        this.f=f;  
    }  
}
```

```
class B extends A {  
    B(){  
        super(0);  
        System.out.println("B constr");  
    }  
}
```

Constructor of A
takes 1 parameter



Possible fix 2: 'super(..)' and 'this(..)'

```
class A {  
    int f;  
    A(int f){  
        this.f=f;  
    }  
}
```

```
class B extends A {  
    B(int f){  
        super(f);  
        System.out.println("B constr");  
    }  
    B(){ this(0); }  
}
```

Constructor of A
takes 1 parameter

Using 'this' instead of 'super'
calls another constructor
in the same class

What Java adds to my code?

- If a class declaration does not use 'extends',
 - it implicitly 'extends Object'
- If a class does not define any constructor
 - It implicitly defines the following: `public ClassName(){ super(); }`
- If a constructor body does not start with 'super(' or 'this(', then a first line of form `super();` is implicitly added.
- Field accesses and method calls on 'this' are automatically recognized (in some cases), and `this.` is implicitly added.
- More additions about generics and nested classes (later in the course)

Is this 'convenience' worth the 'confusion'?

- No. Those 'convenience features' are source of errors and headaches for many expert programmers.
- At the time they did Java (1996) we used to think it was a good idea. It was not.

Java was made with the best intentions,.. but still the result...

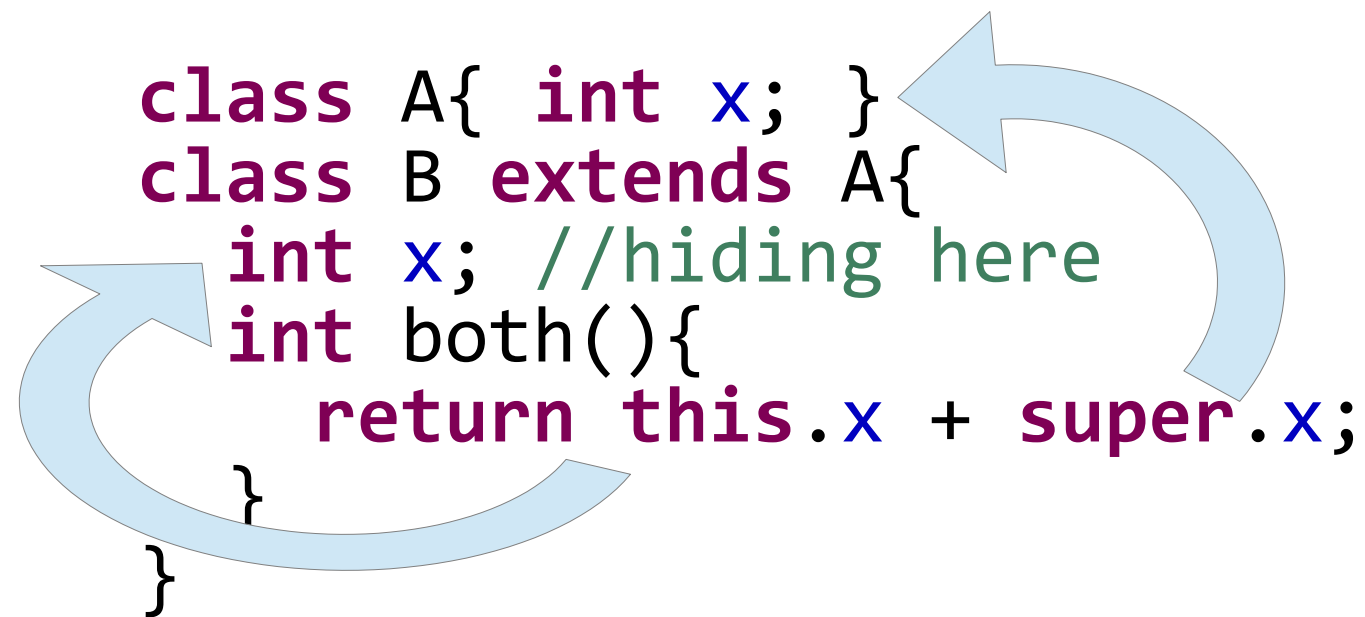
- **Java hates you.**

However, Java it is still the easiest language to learn (fully) by far. That is, other languages may be easier to 'start with', but not to 'fully tame'.

Super

- The syntax “super.” can be used to access a member from the super interface.
- It works for fields too, but it is not used very often

```
class A{ int x; }  
class B extends A{  
    int x; //hiding here  
    int both(){  
        return this.x + super.x;  
    }  
}
```



- Main use case: to call the super implementation of a method!

Super

- We can use “super.” to call the super implementation.
- It is quite common that when overriding a method we want to delegate to the existing implementation, but with a twist;

```
class RoadToll{  
    /*...*/  
    int payFee(){ /*...*/ }  
}
```

```
class RoadTollWithLog extends RoadToll{  
    List<String> log= new ArrayList<>();  
    @Override int payFee(){  
        log.add("PayingFee on: " + LocalDateTime.now());  
        var res= super.payFee();/*...*/  
        log.add("Payed: " + res);  
        return res;  
    }  
}
```

Quiz!

```
class Pow{
    public int pow(int base, int exp){
        if (exp == 0){ return 1; }
        return base * pow(base, exp - 1);
    }
}

class PowLog extends Pow{
    public int pow(int base, int exp){
        System.out.println("LogMessage");
        return super.pow(base, exp);
    }
}

...
System.out.println( new PowLog().pow(4,3) );
```

A) LogMessage
64

LogMessage
LogMessage

B) LogMessage
64

Quiz!

```
class Pow{
    public int pow(int base, int exp){
        if (exp == 0){ return 1; }
        return base * this.pow(base, exp - 1);
    }
}

class PowLog extends Pow{
    public int pow(int base, int exp){
        System.out.println("LogMessage");
        return super.pow(base, exp);
    }
}

...
System.out.println( new PowLog().pow(4,3) );
```

- A) LogMessage
64 
- LogMessage
LogMessage
- B) LogMessage
64 

Supercall for interface default methods

- We can, but we have to specify the interface name as below:

```
interface A{  
    default String mA(){ return "A"; }  
}
```

```
interface B extends A{  
    default String mA(){ return "NotA"; }  
    default String mB(){ return "B" + A.super.mA(); }  
}
```

```
record C() implements B{ // 'implements A, B' would behave identically  
    public String mA(){ return "C" + B.super.mA(); }  
}
```

```
System.out.println( new C().mA() ); //CNotA
```

```
System.out.println( new C().mB() ); //BA
```

```
//Note: we can 'access' the implementation  
//of A by relying on the super of B.
```

```
//This is often needed (forwarding a super)
```

The background of the image features two Super Saiyan Goku characters from the Dragon Ball series. They are depicted in a dynamic, battle-ready pose, facing each other with their arms outstretched. The character on the left is wearing his signature orange gi with blue trim and a blue sash. The character on the right is wearing a dark blue gi with orange trim and an orange sash. Both characters have spiky black hair and are surrounded by intense, glowing energy auras in shades of blue and yellow. The overall scene is set against a dark, rocky landscape with a bright, glowing light source in the background, creating a dramatic and high-energy atmosphere.

abstract classes vs interfaces

Abstract classes vs interfaces!

- From Java8, both Abstract classes and interfaces can be used for code reuse.
- Both can not be instantiated
- They both can contain abstract methods as well as implemented/default methods.
 - Abstract methods have no implementation
 - Concrete subclasses must provide it
- Abstract classes may also contain (private) fields and constructors
- Interfaces can have abstract getters and setters to “imagine” the presence of fields.
- When field+constructors are not strictly needed,
interfaces should be used instead of abstract classes

Abstract classes vs interfaces

- Interfaces:
- Abstract classes:

Abstract classes vs interfaces

- Interfaces:
~~instance fields~~
- Abstract classes:
instance fields

Note: interfaces can have static final public fields, using syntax

`Type fieldName= expression;`

Note that this looks a lot like a normal instance field...

Java hates you.

Abstract classes vs interfaces

- Interfaces:

~~instance fields~~

~~constructors~~

- Abstract classes:

instance fields

constructors

Abstract classes vs interfaces

- Interfaces:

~~instance fields~~

~~constructors~~

~~privates~~

- Abstract classes:

instance fields

constructors

privates

Note: private methods and private static methods are allowed, but

- no private fields (since no abstract private methods)
- no private static fields (but yes to static and private static methods)
- no private nested classes (as we will see later) (*except with tricks)

Abstract classes vs interfaces

- Interfaces:

~~instance fields~~

~~constructors~~

~~privates~~

many!

- Abstract classes:

instance fields

constructors

privates

~~many~~

- All abstract classes with no data properties/invariants
(as in `assert !name.isEmpty();`)
can be replaced it with interfaces to improve code reuse.

The perfect case

- What is the perfect case to use an interface?
- What is the perfect case to use an abstract class?



The perfect case to use an abstract class

- The abstract class represents a set of operations working together to a goal.
- The various methods are pieces of those operations, that can be independently personalized.
- Those operation depends on some state that must be managed by the operations and **not by the user code**.
- That is, the abstract class code is the only responsible to
 - initialize this state
 - read/update this state

A reusable solid foundation with state invariants can be provided by abstract classes



The perfect case to use an abstract class

```
abstract class LibraryProperty{  
    private final static List<LibraryProperty> all= new ArrayList<>();  
    public LibraryProperty(){  
        all.add(this);  
    }  
    public static List<LibraryProperty> inventory(){  
        return Collections.unmodifiableList(all);  
    }  
}
```

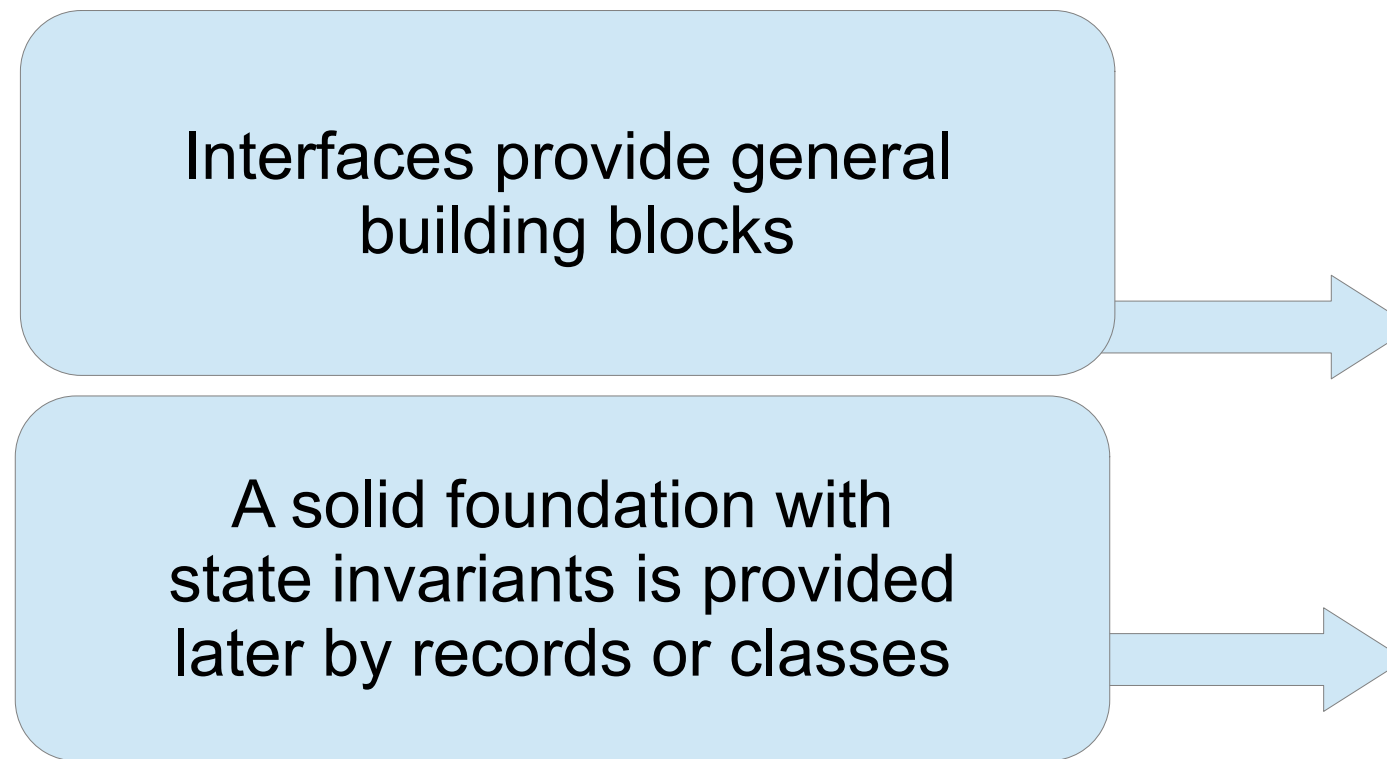
```
class Book extends LibraryProperty{}  
//new Book() will automatically add it to the list!  
class Magazine extends LibraryProperty{}  
//new Magazine() will automatically add it to the list!  
class Newspaper extends LibraryProperty{}  
//new Newspaper() will automatically add it to the list!
```

The perfect case to use an abstract class

```
abstract class Animal{
    private String name;
    private int age;
    public Animal(String name, int age){
        name(name);
        age(age);
    }
    final public String name(){ return name; }
    final public int age(){ return age; }
    final public void name(String v){
        if ( v == null || v.isBlank()){ throw new IllegalArgumentException(); }
        name = v.trim();
    }
    final public void age(int v){
        if (v < 0 || v > 1000){ throw new IllegalArgumentException(); }
        age = v;
    }
}
class Cat extends Animal{//all subclasses now guarantee properties on the state
    public Cat(String name, int age){ super(name,age); }
    //...
}
```

The perfect case to use an interface

- The interface represents a set of operations working together to a goal.
- The various interface methods are pieces of those operations, that can be independently personalized.
- Those operation are either stateless or depend on state that can be initialized and observed by the USER of the operation.



The perfect case to use an interface

All concrete
subclasses
of length
must have
meters() and
yards()
methods

```
interface Length{  
    default double meters(){ return yards() * 0.91d; }  
    default double yards(){ return meters() * 1.09d; }  
    default Length add(Length l){  
        return new Meters(l.meters() + this.meters());  
    }  
}  
  
record Yards(double yards) implements Length{}  
  
record Meters(double meters) implements Length{}
```

The field name is very relevant here: it generates
a method meters() that overrides Length.meters() !!

Combining implementations!

```
interface Chest{  
    List<Item> get();  
    default void depositItem(Item i){/*...*/}  
}
```



Combining implementations!

```
interface Chest{  
    List<Item> get();  
    default void depositItem(Item i){/*...*/}  
}
```

```
interface Minecart{  
    Point position();  
    void position(Point val);  
    default void move(Map map){/*...*/}  
}
```



Combining implementations!

```
interface Chest{  
    List<Item> get();  
    default void depositItem(Item i){/*...*/}  
}
```

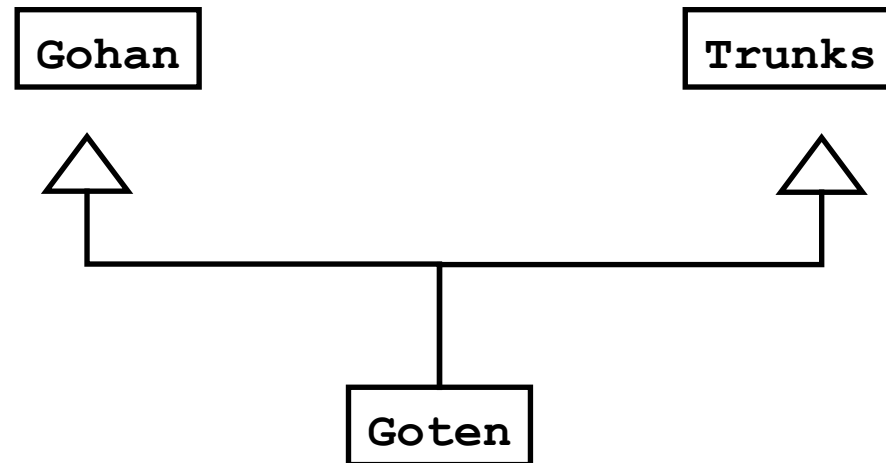
```
interface Minecart{  
    Point position();  
    void position(Point val);  
    default void move(Map map){/*...*/}  
}
```

```
class MinecartChest implements Chest, Minecart{  
    Point position;  
    List<Item> items= new ArrayList<>();  
    public MinecartChest(Point p){ position(p); }  
    public List<Item> get(){ return this.items; }  
    public Point position(){ return this.position; }  
    public void position(Point val){ this.position = val; }  
}
```



Multiple inheritance

<https://www.youtube.com/watch?v=8cpyejx506o>



- From Java8, this is now possible!
 - A class cannot have more than one superclass
 - Other languages (e.g. C++) support this
 - But, a class can implement **more than one interface**
 - Interfaces can now have reusable implementation!

```
class Goten extends Gohan, Trunks { ... }
```



```
class Goten implements Gohan, Trunks { ... }
```



- Risk of multiple inheritance: we may end up with too much stuff,
- A class that is too fat.
(violating the single responsibility principle)
- To avoid this, we need to make the interfaces used to modularize our behaviour as slim as possible



- To avoid this, we need to make the interfaces used to modularize our behaviour as slim as possible
- We can have many different interfaces, sometimes with just a single default method (and the private methods that it uses internally)
- The other methods will be abstract and defined in other interfaces.



Multiple inheritance

- No fields – so no need of ‘precise initialization order once and only once’
 - Initialization of state is deferred to the implementing class/record
- Only methods – if the method bodies are conflicting we can
 - Override them with new implementation
 - Override them by calling `SomeInterfaceName.super.methName(...)`;
 - Override them by declaring them abstract.



An aerial photograph of a slum area. In the foreground, several buildings with bright blue and green corrugated metal roofs are visible. To the right, a building with a white and brown corrugated metal roof is in a state of disrepair, with debris scattered around it. The background shows a paved road and more buildings. The text "Next The ugly parts of old Java" is overlaid in the center.

Next The ugly parts of old Java